Linux x86-64 System Call Cheat Sheet

C-style synopses + syscall numbers • Native x86-64 Linux ABI • Study reference

Scope and terminology

Linux system calls are kernel entry points. The C prototypes below are the user-space interfaces documented by Linux man-pages (often libc wrappers); they are not always identical to the raw kernel ABI. Some calls have architecture-specific argument layouts, and some libc functions use a different syscall internally. The number index is for native x86-64, not i386 or x32.

Raw x86-64 syscall register order

Register	Meaning
rax	System-call number; return value after syscall
rdi	Argument 1
rsi	Argument 2
rdx	Argument 3
r10	Argument 4
r8	Argument 5
r9	Argument 6

Important: the fourth raw syscall argument is in `r10`, not `rcx`. The `syscall` instruction clobbers `rcx` and `r11`. Raw syscall errors are returned as negative error numbers in `rax`; libc wrappers generally return -1 and set `errno`.

Common and useful syscall synopses

Grouped for assembly practice, Linux programming, debugging, and reverse engineering. The syscall number column refers to native x86-64.

File I/O and descriptors

No.	Call	C synopsis	What it does
0	read	ssize_t read(int fd, void *buf, size_t count);	Read up to count bytes from a file descriptor.
1	write	ssize_t write(int fd, const void *buf, size_t count);	Write up to count bytes to a file descriptor.
2	open	int open(const char *pathname, int flags, ... /* mode_t mode */);	Open or create a file. mode is supplied when creating.
3	close	int close(int fd);	Close a file descriptor.
8	lseek	off_t lseek(int fd, off_t offset, int whence);	Reposition a file offset.
17	pread64	ssize_t pread(int fd, void *buf, size_t count, off_t offset);	Read at an offset without changing the file offset.
18	pwrite64	ssize_t pwrite(int fd, const void *buf, size_t count, off_t offset);	Write at an offset without changing the file offset.

No.	Call	C synopsis	What it does
19	readv	ssize_t readv(int fd, const struct iovec *iov, int iovcnt);	Read into multiple buffers.
20	writev	ssize_t writev(int fd, const struct iovec *iov, int iovcnt);	Write from multiple buffers.
21	access	int access(const char *pathname, int mode);	Check real-UID/GID access permissions.
22	pipe	int pipe(int pipefd[2]);	Create a unidirectional pipe.
32	dup	int dup(int oldfd);	Duplicate a file descriptor.
33	dup2	int dup2(int oldfd, int newfd);	Duplicate onto a specified descriptor.
72	fcntl	int fcntl(int fd, int op, ...);	Operate on a descriptor or its open-file description.
74	fsync	int fsync(int fd);	Flush file data and metadata to storage.
75	fdatasync	int fdatasync(int fd);	Flush file data and metadata needed for retrieval.
217	getdents64	ssize_t getdents64(unsigned int fd, struct linux_dirent64 *dirp, unsigned int count);	Read directory entries (Linux-specific; usually called via syscall).
257	openat	int openat(int dirfd, const char *pathname, int flags, ... /* mode_t mode */);	Open relative to a directory descriptor.
262	newfstatat	int fstatat(int dirfd, const char *restrict path, struct stat *restrict statbuf, int flags);	Get file metadata relative to a directory descriptor.
293	pipe2	int pipe2(int pipefd[2], int flags);	Create a pipe with flags such as O_CLOEXEC.
292	dup3	int dup3(int oldfd, int newfd, int flags);	Duplicate a descriptor with flags.

Memory management

No.	Call	C synopsis	What it does
9	mmap	void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);	Map files or anonymous memory.
10	mprotect	int mprotect(void *addr, size_t len, int prot);	Change memory-page protections.
11	munmap	int munmap(void *addr, size_t length);	Unmap a memory region.
12	brk	int brk(void *addr);	Change the process data-segment break (libc wrapper semantics).
25	mremap	void *mremap(void *old_address, size_t old_size, size_t new_size, int flags, ...);	Resize or move a mapping.
26	msync	int msync(void *addr, size_t length, int flags);	Synchronize a mapped region with its backing file.
28	madvise	int madvise(void *addr, size_t length, int advice);	Give the kernel advice about memory usage.
462	mseal	int mseal(void *addr, size_t size, unsigned long flags);	Seal a virtual-memory range against selected changes.

Processes, execution, and signals

No.	Call	C synopsis	What it does
56	clone	long clone(unsigned long flags, void *stack, int *parent_tid, int *child_tid, unsigned long tls);	Create a process/thread with selected sharing semantics (raw ABI differs by architecture).
57	fork	pid_t fork(void);	Create a child process.
58	vfork	pid_t vfork(void);	Create a child with temporary address-space sharing.
59	execve	int execve(const char *pathname, char *const argv[], char *const envp[]);	Replace the current process image.
60	exit	void _exit(int status);	Terminate the calling process (libc wrapper may use exit_group).
231	exit_group	void _exit(int status);	Terminate all threads in the process (raw syscall).
61	wait4	pid_t wait4(pid_t pid, int *wstatus, int options, struct rusage *rusage);	Wait for child state changes and collect usage.
62	kill	int kill(pid_t pid, int sig);	Send a signal to a process or process group.
13	rt_sigaction	int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);	Examine/change signal disposition (libc wrapper).
14	rt_sigprocmask	int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);	Examine/change blocked signal mask.
101	ptrace	long ptrace(enum __ptrace_request request, pid_t pid, void *addr, void *data);	Trace/control another process; central to debuggers.
157	prctl	int prctl(int op, unsigned long arg2, unsigned long arg3, unsigned long arg4, unsigned long arg5);	Process/thread-specific control operations.
158	arch_prctl	int arch_prctl(int code, unsigned long addr);	Set/get architecture-specific thread state (notably FS/GS base).
39	getpid	pid_t getpid(void);	Get the calling process ID.
186	gettid	pid_t gettid(void);	Get calling thread ID (Linux-specific).
434	pidfd_open	int pidfd_open(pid_t pid, unsigned int flags);	Obtain a file descriptor referring to a process.
424	pidfd_send_signal	int pidfd_send_signal(int pidfd, int sig, siginfo_t *info, unsigned int flags);	Send a signal through a process descriptor.
322	execveat	int execveat(int dirfd, const char *pathname, char *const argv[], char *const envp[], int flags);	Execute a file relative to a directory descriptor.

Directories and metadata

No.	Call	C synopsis	What it does
4	stat	int stat(const char *restrict path, struct stat *restrict statbuf);	Get file metadata by pathname.
5	fstat	int fstat(int fd, struct stat *statbuf);	Get metadata from an open descriptor.
6	lstat	int lstat(const char *restrict path, struct stat *restrict statbuf);	Get metadata without following a final symlink.
79	getcwd	char *getcwd(char *buf, size_t size);	Get current working directory.
80	chdir	int chdir(const char *path);	Change current working directory.
83	mkdir	int mkdir(const char *pathname, mode_t mode);	Create a directory.

No.	Call	C synopsis	What it does
84	rmdir	int rmdir(const char *pathname);	Remove an empty directory.
85	creat	int creat(const char *pathname, mode_t mode);	Create or truncate a file (equivalent to open with flags).
87	unlink	int unlink(const char *pathname);	Remove a directory entry.
89	readlink	ssize_t readlink(const char *restrict path, char *restrict buf, size_t bufsiz);	Read the contents of a symbolic link.
90	chmod	int chmod(const char *pathname, mode_t mode);	Change file permission bits.
95	umask	mode_t umask(mode_t mask);	Set the process file-creation mask.
332	statx	int statx(int dirfd, const char *pathname, int flags, unsigned int mask, struct statx *statxbuf);	Retrieve extended file metadata.
263	unlinkat	int unlinkat(int dirfd, const char *pathname, int flags);	Remove a path relative to a directory descriptor.

Networking

No.	Call	C synopsis	What it does
41	socket	int socket(int domain, int type, int protocol);	Create a socket.
42	connect	int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);	Connect a socket to a peer.
43	accept	int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);	Accept a connection on a listening socket.
44	sendto	ssize_t sendto(int sockfd, const void *buf, size_t len, int flags, const struct sockaddr *dest_addr, socklen_t addrlen);	Send a message, optionally to a destination.
45	recvfrom	ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr *src_addr, socklen_t *addrlen);	Receive a message and optionally its source address.
46	sendmsg	ssize_t sendmsg(int sockfd, const struct msghdr *msg, int flags);	Send a message described by msghdr.
47	recvmsg	ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);	Receive a message described by msghdr.
48	shutdown	int shutdown(int sockfd, int how);	Shut down part or all of a full-duplex connection.
49	bind	int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);	Assign a local address to a socket.
50	listen	int listen(int sockfd, int backlog);	Mark a stream socket as passive/listening.
288	accept4	int accept4(int sockfd, struct sockaddr *addr, socklen_t *addrlen, int flags);	Accept a socket with flags.

Timing, synchronization, and event loops

No.	Call	C synopsis	What it does
35	nanosleep	int nanosleep(const struct timespec *req, struct timespec *rem);	Sleep for a specified interval.
228	clock_gettime	int clock_gettime(clockid_t clockid, struct timespec *tp);	Read a clock.
202	futex	int futex(uint32_t *uaddr, int futex_op, uint32_t val, ...);	Low-level userspace synchronization primitive.

No.	Call	C synopsis	What it does
24	sched_yield	int sched_yield(void);	Yield the processor to another runnable thread.
7	poll	int poll(struct pollfd *fds, nfds_t nfds, int timeout);	Wait for events on file descriptors.
23	select	int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);	Monitor sets of file descriptors.
291	epoll_create1	int epoll_create1(int flags);	Create an epoll instance.
233	epoll_ctl	int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);	Add, modify, or remove an epoll interest.
232	epoll_wait	int epoll_wait(int epfd, struct epoll_event *events, int maxevents, int timeout);	Wait for epoll events.

Security, randomness, and newer interfaces

No.	Call	C synopsis	What it does
318	getrandom	ssize_t getrandom(void *buf, size_t buflen, unsigned int flags);	Obtain bytes from the kernel random source.
319	memfd_create	int memfd_create(const char *name, unsigned int flags);	Create an anonymous RAM-backed file descriptor.
317	seccomp	int seccomp(unsigned int operation, unsigned int flags, void *args);	Configure secure-computing syscall filters.
321	bpf	int bpf(int cmd, union bpf_attr *attr, unsigned int size);	Interact with the BPF subsystem.
425	io_uring_setup	int io_uring_setup(unsigned int entries, struct io_uring_params *p);	Create an io_uring submission/completion queue.

Numerical syscall order — native x86-64

This is the number-first lookup sheet: find the number from `rax`, then identify the syscall name. Blank numbers are reserved or unused; the x32 ABI uses a separate numbering range. The list is generated from the x86-64 syscall header available when this sheet was built, with newer published entries appended where applicable.

Number	Syscall	Number	Syscall	Number	Syscall
0	read	1	write	2	open
3	close	4	stat	5	fstat
6	lstat	7	poll	8	lseek
9	mmap	10	mprotect	11	munmap
12	brk	13	rt_sigaction	14	rt_sigprocmask
15	rt_sigreturn	16	ioctl	17	pread64
18	pwrite64	19	readv	20	writev
21	access	22	pipe	23	select
24	sched_yield	25	mremap	26	msync
27	mincore	28	madvise	29	shmget
30	shmat	31	shmctl	32	dup
33	dup2	34	pause	35	nanosleep
36	getitimer	37	alarm	38	setitimer
39	getpid	40	sendfile	41	socket
42	connect	43	accept	44	sendto
45	recvfrom	46	sendmsg	47	recvmsg
48	shutdown	49	bind	50	listen
51	getsockname	52	getpeername	53	socketpair
54	setsockopt	55	getsockopt	56	clone
57	fork	58	vfork	59	execve
60	exit	61	wait4	62	kill
63	uname	64	semget	65	semop
66	semctl	67	shmdt	68	msgget
69	msgsnd	70	msgrcv	71	msgctl
72	fcntl	73	flock	74	fsync
75	fdatasync	76	truncate	77	ftruncate
78	getdents	79	getcwd	80	chdir
81	fchdir	82	rename	83	mkdir

Number	Syscall	Number	Syscall	Number	Syscall
84	rmdir	85	creat	86	link
87	unlink	88	symlink	89	readlink
90	chmod	91	fchmod	92	chown
93	fchown	94	lchown	95	umask
96	gettimeofday	97	getrlimit	98	getrusage
99	sysinfo	100	times	101	ptrace
102	getuid	103	syslog	104	getgid
105	setuid	106	setgid	107	geteuid
108	getegid	109	setpgid	110	getppid
111	getpgrp	112	setsid	113	setreuid
114	setregid	115	getgroups	116	setgroups
117	setresuid	118	getresuid	119	setresgid
120	getresgid	121	getpgid	122	setfsuid
123	setfsgid	124	getsid	125	capget
126	capset	127	rt_sigpending	128	rt_sigtimedwait
129	rt_sigqueueinfo	130	rt_sigsuspend	131	sigaltstack
132	utime	133	mknod	134	uselib
135	personality	136	ustat	137	statfs
138	fstatfs	139	sysfs	140	getpriority
141	setpriority	142	sched_setparam	143	sched_getparam
144	sched_setscheduler	145	sched_getscheduler	146	sched_get_priority_max
147	sched_get_priority_min	148	sched_rr_get_interval	149	mlock
150	munlock	151	mlockall	152	munlockall
153	vhangup	154	modify_ldt	155	pivot_root
156	_sysctl	157	prctl	158	arch_prctl
159	adjtimex	160	setrlimit	161	chroot
162	sync	163	acct	164	settimeofday
165	mount	166	umount2	167	swapon
168	swapoff	169	reboot	170	sethostname
171	setdomainname	172	iopl	173	ioperm
174	create_module	175	init_module	176	delete_module

Number	Syscall	Number	Syscall	Number	Syscall
177	get_kernel_syms	178	query_module	179	quotactl
180	nfsservctl	181	getpmsg	182	putpmsg
183	afs_syscall	184	tuxcall	185	security
186	gettid	187	readahead	188	setxattr
189	lsetxattr	190	fsetxattr	191	getxattr
192	lgetxattr	193	fgetxattr	194	listxattr
195	llistxattr	196	flistxattr	197	removexattr
198	lremovexattr	199	fremovexattr	200	tkill
201	time	202	futex	203	sched_setaffinity
204	sched_getaffinity	205	set_thread_area	206	io_setup
207	io_destroy	208	io_getevents	209	io_submit
210	io_cancel	211	get_thread_area	212	lookup_dcookie
213	epoll_create	214	epoll_ctl_old	215	epoll_wait_old
216	remap_file_pages	217	getdents64	218	set_tid_address
219	restart_syscall	220	semtimedop	221	fadvise64
222	timer_create	223	timer_settime	224	timer_gettime
225	timer_getoverrun	226	timer_delete	227	clock_settime
228	clock_gettime	229	clock_getres	230	clock_nanosleep
231	exit_group	232	epoll_wait	233	epoll_ctl
234	tgkill	235	utimes	236	vserver
237	mbind	238	set_mempolicy	239	get_mempolicy
240	mq_open	241	mq_unlink	242	mq_timedsend
243	mq_timedreceive	244	mq_notify	245	mq_getsetattr
246	kexec_load	247	waitid	248	add_key
249	request_key	250	keyctl	251	ioprio_set
252	ioprio_get	253	inotify_init	254	inotify_add_watch
255	inotify_rm_watch	256	migrate_pages	257	openat
258	mkdirat	259	mknodat	260	fchownat
261	futimesat	262	newfstatat	263	unlinkat
264	renameat	265	linkat	266	symlinkat
267	readlinkat	268	fchmodat	269	faccessat

Number	Syscall	Number	Syscall	Number	Syscall
270	pselect6	271	ppoll	272	unshare
273	set_robust_list	274	get_robust_list	275	splice
276	tee	277	sync_file_range	278	vmsplice
279	move_pages	280	utimensat	281	epoll_pwait
282	signalfd	283	timerfd_create	284	eventfd
285	fallocate	286	timerfd_settime	287	timerfd_gettime
288	accept4	289	signalfd4	290	eventfd2
291	epoll_create1	292	dup3	293	pipe2
294	inotify_init1	295	preadv	296	pwritev
297	rt_tgsigqueueinfo	298	perf_event_open	299	recvmmsg
300	fanotify_init	301	fanotify_mark	302	prlimit64
303	name_to_handle_at	304	open_by_handle_at	305	clock_adjtime
306	syncfs	307	sendmmsg	308	setns
309	getcpu	310	process_vm_readv	311	process_vm_writev
312	kcmp	313	finit_module	314	sched_setattr
315	sched_getattr	316	renameat2	317	seccomp
318	getrandom	319	memfd_create	320	kexec_file_load
321	bpf	322	execveat	323	userfaultfd
324	membarrier	325	mlock2	326	copy_file_range
327	preadv2	328	pwritev2	329	pkey_mprotect
330	pkey_alloc	331	pkey_free	332	statx
333	io_pgetevents	334	rseq	335	uretprobe
424	pidfd_send_signal	425	io_uring_setup	426	io_uring_enter
427	io_uring_register	428	open_tree	429	move_mount
430	fsopen	431	fsconfig	432	fsmount
433	fspick	434	pidfd_open	435	clone3
436	close_range	437	openat2	438	pidfd_getfd
439	faccessat2	440	process_madvise	441	epoll_pwait2
442	mount_setattr	443	quotactl_fd	444	landlock_create_ruleset
445	landlock_add_rule	446	landlock_restrict_self	447	memfd_secret
448	process_mrelease	449	futex_waitv	450	set_mempolicy_home_node

Number	Syscall	Number	Syscall	Number	Syscall
451	cachestat	452	fchmodat2	453	map_shadow_stack
454	futex_wake	455	futex_wait	456	futex_requeue
457	statmount	458	listmount	459	lsm_get_self_attr
460	lsm_set_self_attr	461	lsm_list_modules	462	mseal
463	setxattrat	464	getxattrat	465	listxattrat
466	removexattrat	467	open_tree_attr	468	file_getattr
469	file_setattr	470	listns	471	rseq_slice_yield
472	fchroot				

Quick assembly example: write(1, buf, 5)

```
mov rax, 1 ; __NR_write
mov rdi, 1 ; stdout
mov rsi, buf ; pointer to bytes
mov rdx, 5 ; byte count
syscall
```

Reference sources

Linux kernel x86-64 syscall table: https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl
Linux syscall(2): https://man7.org/linux/man-pages/man2/syscall.2.html
Linux man-pages project: https://www.kernel.org/doc/man-pages/

Version note: syscall numbers can grow as the kernel adds interfaces. Check the syscall table and your target system's headers for the exact kernel/architecture version. Some historical entries are obsolete, reserved, or not implemented on current kernels.